

Technical_Learning_List

Technical Learning List

LastOne — What to get up to speed on before and during the build

Compiled: May 2026

The Architecture Guide covers the conceptual foundation: three-layer model, APIs, auth, state, React Native primitives, Flexbox, Expo Router, TypeScript basics, Supabase (Auth, RLS, Realtime, Edge Functions), monorepo structure, folder structure, and the build/deploy workflow.

Everything below is what the Architecture Guide does not cover — organised by category and prioritised by when you will need it.

Priority 1 — Before Build Starts

These are the things you need to understand before writing the first feature. Not knowing them will block you or send you in the wrong direction from day one.

React Query (TanStack Query)

The standard library for managing server state in React. The Architecture Guide names it but doesn't explain it. You need to understand:

- `useQuery` — fetches data and keeps it cached
- `useMutation` — writes data (inserts, updates, deletes)
- Query keys — how React Query identifies and invalidates cached data
- Cache invalidation — how to tell the app "this data is stale, refetch it" after a mutation

Every screen that reads from Supabase will use `useQuery`. Every action that writes to Supabase will use `useMutation`. This is not optional knowledge.

React Context

How global state (specifically auth state) flows through the entire app. The Architecture Guide mentions it briefly. You need to understand:

- How to create a context and a provider
- How to wrap the root of the app in that provider
- How to consume the context from any screen using `useContext`

The auth state check in `app/_layout.tsx` — which decides whether to show the login screen or the main app — is built entirely on React Context.

NativeWind (in depth)

The Architecture Guide lists it in the tech stack but doesn't explain it. NativeWind brings Tailwind CSS utility classes to React Native. You need to understand:

- How Tailwind utility classes work (`flex-1`, `bg-white`, `text-lg`, `p-4`, etc.)
- How to configure the theme in `tailwind.config.js` — this is where design tokens live
- How to add custom values (your specific colours, font sizes, spacing) to the config
- How styled components work in NativeWind v4

This is the styling system for the entire app. Agree on design tokens with your design team and put them in this config file before any screen is built.

Supabase CLI

Used daily. The guide mentions it once for type generation. You need to know how to:

- Run `supabase init` and `supabase start` to run a local Supabase instance for development
- Run `supabase db push` to apply migrations to your live database
- Run `supabase gen types typescript` to auto-generate TypeScript types from your schema
- Run `supabase functions serve` to test Edge Functions locally before deploying

SQL for Migrations

The guide explains what migrations are. You need to know enough SQL to read and review what Claude Code generates — not write it from scratch:

- `CREATE TABLE` — defining tables and columns
- Foreign key constraints — linking tables together
- `ALTER TABLE` — adding columns to an existing table
- RLS policy syntax — `CREATE POLICY`, `USING`, `WITH CHECK`
- Basic indexes — `CREATE INDEX ON table (column)`

You will catch mistakes in Claude Code's output if you understand these. You will miss them if you don't.

Debugging React Native

Not covered anywhere. When something breaks, you need to know how to find out why:

- Metro bundler logs — your terminal where `npx expo start` is running shows errors

- `console.log` — output appears in the Metro terminal, not the browser
- Expo DevTools — opens in the browser, shows device logs
- React Native Debugger — standalone app for stepping through component state and network requests
- Supabase logs — in the Supabase dashboard under Logs, shows failed API calls and Edge Function output

This is day-one operational knowledge. Set up your debugging workflow before you write your first screen.

Priority 2 — Learn As You Hit Them

These come up during specific features in the build sequence. Learn each one when you reach that feature rather than all upfront.

Keyboard Handling (*hits you at: Onboarding, Logging*)

React Native has specific patterns for dealing with the on-screen keyboard covering input fields. Libraries: `KeyboardAvoidingView` (built-in) and `react-native-keyboard-aware-scroll-view`. You will hit this on the first day of building Onboarding.

React Context for Feature State (*hits you at: Home Screen, throughout*)

Beyond auth, you may use Context for other global state (user profile data, stage). The same pattern applies — create, provide, consume.

Reanimated 2 — React Native Animations (*hits you at: Coping Library, Streak display*)

The standard library for smooth animations in React Native. Relevant for: the breathing tool sequences, streak counter animations, milestone celebrations. You don't need to master it — Claude Code handles the syntax — but you need conceptual understanding to direct it and review output. Key concepts: `useSharedValue`, `useAnimatedStyle`, `withTiming`, `withSpring`.

Gesture Handling (*hits you at: Coping Library*)

`react-native-gesture-handler` — for swipe gestures and interactive sliders in the coping tools. Conceptual understanding is enough.

Push Notifications in Depth (*hits you at: Notifications — build step 8*)

`expo-notifications` is mentioned in the Architecture Guide but the full picture is more complex than it appears:

- Getting an **Expo Push Token** from the device — this is the device's unique address for notifications
- Storing that token per user in Supabase — each user can have multiple devices
- A **trigger mechanism** (Edge Function or client-side logic) that sends the notification at the right time
- The **permission request flow** — asking the user to allow notifications, handling graceful denial
- **Notification handlers** — what happens when a notification arrives while the app is open vs closed

This is one of the more involved features in the build. Give it more time than you'd expect.

App Permissions (*hits you at: Notifications, potentially camera for profile photos*)

React Native has an explicit permissions model. Key things to understand:

- iOS asks for permission once and remembers the answer permanently
- Android has different permission tiers depending on version
- You have to handle the case where the user denies permission — the app should still work, just without that feature
- `expo-permissions` or the individual Expo module handles the request

Offline Patterns (*hits you at: Home Screen, Streak System*)

LastOne's streak counter must work without internet. The full offline pattern:

- `expo-secure-store` — for caching the user session and critical data (quit_date, current streak) locally
- `@react-native-community/netinfo` — detecting whether the device is online or offline
- Displaying cached data when offline rather than an error screen
- Queueing writes (craving logs entered offline) and syncing when connectivity returns — this is a more advanced pattern, defer to V2 if needed

Edge Functions in Depth (*hits you at: Notifications, optionally Streak validation*)

The guide explains what they are. Writing and deploying one requires:

- Understanding the **Deno runtime** — Edge Functions run on Deno, not Node.js. Most JavaScript/TypeScript works the same way, but module imports use URLs rather than npm packages

- `supabase functions new` — scaffolds a new function
- `supabase functions serve` — runs it locally for testing
- `supabase functions deploy` — deploys to production
- **Scheduled functions** — triggered by a cron schedule via Supabase's `pg_cron` extension, not on demand

Image Handling (*hits you at: Profile screen*)

For user profile photos:

- `expo-image-picker` — lets the user select a photo from their camera roll or take one with the camera
- Supabase Storage upload — sending the image to your Supabase storage bucket
- Serving it back — Supabase generates a public URL for the image that you store on the user's profile record
- Image resizing — consider resizing before upload to keep storage costs low; `expo-image-manipulator` handles this

Database Indexing (*hits you at: any point where queries get slow*)

Not mentioned anywhere in the current docs. As `craving_logs` grows (potentially thousands of rows per user), queries against it without proper indexes will get slow. You need to understand:

- What an index is — a lookup structure that makes queries on a column fast
- When to add one — any column you frequently filter or sort by (`user_id` , `logged_at` , `stage`)
- How to add one — `CREATE INDEX ON craving_logs (user_id, logged_at DESC)`
- Claude Code will not always add these automatically — you need to ask or spot the gap

Priority 3 — Good to Know, Learn on Demand

These are not blockers. Learn them when a specific need arises.

Error Handling Patterns

What happens when a Supabase query fails? React Native has no default error UI. You need `try/catch` with `async/await` and a pattern for surfacing errors to the user gracefully (a toast, an inline message, a retry button). Not complex, but think about it before you build your first data-fetching screen.

Supabase Realtime in Depth

The Architecture Guide explains what Realtime is. If you build the social layer in V2, you'll need: channels, subscriptions, presence, and how to clean up subscriptions when a component unmounts (important to prevent memory leaks). Defer this until V2.

Deep Linking

Allows external links to open specific screens in the app. Relevant if you want milestone share cards to link back to the app. Expo Router has built-in support. Defer to V2.

Sentry / Crash Reporting

For a production app with real users, you want to know when the app crashes and why. Sentry is the standard tool and has an Expo SDK. Not needed for a prototype but worth setting up before any real user testing.

The Supabase Dashboard (operational fluency)

Not a technical topic but daily-use knowledge. Get comfortable with:

- **Table Editor** — view and manually edit rows in your database
- **SQL Editor** — run custom queries for debugging
- **Auth** — see all registered users, manage sessions
- **Storage** — manage uploaded files, configure bucket policies
- **Edge Functions** — deploy, view logs, check for errors
- **Logs** — debug API errors, slow queries, Edge Function failures

Summary — by priority

Priority	Topic	When
1	React Query	Before build
1	React Context	Before build
1	NativeWind in depth	Before build
1	Supabase CLI	Before build
1	SQL for migrations	Before build
1	Debugging React Native	Before build
2	Keyboard handling	Onboarding
2	Reanimated 2	Coping Library
2	Gesture handling	Coping Library
2	Push notifications	Notifications feature
2	App permissions	Notifications feature

Priority	Topic	When
2	Offline patterns	Home Screen / Streak
2	Edge Functions in depth	Notifications / Streak
2	Image handling	Profile screen
2	Database indexing	Ongoing
3	Error handling patterns	First data-fetching screen
3	Supabase Realtime	V2
3	Deep linking	V2
3	Sentry / crash reporting	Pre-user testing
3	Supabase dashboard fluency	Ongoing